PROJECT REPORT: Implement Client Script & UI Policy (Incident)

Executive Summary

This project demonstrates the implementation of ServiceNow client-side controls—specifically **UI Policies** and **Client Scripts**—to enforce data integrity, field visibility, conditional mandatory status, and save-time validations on Incident records.

By executing business rules dynamically at the user interface level, the solution eliminates incomplete and inconsistent data entry, thereby enhancing reporting accuracy, SLA compliance, and overall service quality.

---

1. Problem Statement & Objective

- **Problem Statement:** Incident records often suffer from inconsistent and inaccurate data entry when relying solely on manual user checks. Incomplete submissions negatively impact triage efficiency, SLA compliance, and service reporting.
- **Objective:** Establish ServiceNow client-side controls to dynamically set mandatory fields, auto-populate critical values, restrict editing privileges, and validate input before saving or modifying Incident records.

---

2. Core Skills & Concepts

- **ServiceNow Platform:** Incident Management Architecture
- **Form Logic:** UI Policies & UI Policy Actions
- **Scripting Engine:** Client Scripts (`onChange`, `onSubmit`, `onCellEdit`)
- **Validation & Governance:** Form & List View Integrity Verification

---

3. Implementation Tasks

Task 1: Create UI Policy on Incident ("High Impact Control")

- **Navigation:** `System UI` → `UI Policies` → `New`
- **Configuration:**
  - **Short Description:** High Impact Control
  - **Table:** Incident (`incident`)
  - **Active:** `true`
  - **Condition:** `Impact is 1 - High`
  - **Reverse if false:** `true`

UI Policy Action (Assignment Group):

- **Field Name:** `Assignment group`
- **Mandatory:** `True`
- **Read-only:** `False`

Task 2: Create UI Policy Action for Urgency Field

Configuration under "High Impact Control" UI Policy:

- **Field Name:** `Urgency`
- **Read-only:** `True`
- **Visible:** `Leave alone`

Task 3: Create onChange Client Script (Auto Set Urgency)

- **Navigation:** `System UI` → `Client Scripts` → `New`
- **Configuration:**
  - **Name:** Auto set urgency for high impact
  - **Table:** Incident (`incident`)
  - **Type:** `onChange`
  - **Field Name:** `Impact`

- **Active:** `true`

Script Logic:function onChange(control, oldValue, newValue, isLoading) {

if (isLoading || newValue == '') {

|---|

}

if (newValue == '1') {

g_form.setValue('urgency', '1');

g_form.addInfoMessage('Urgency set to High for High impact incident.');

}

}

Task 4: Create `onSubmit` Client Script (Save Validation)

- **Navigation:** `System UI` → `Client Scripts` → `New`
- **Configuration:**
  - **Name:** Prevent save if Assigned To missing
  - **Table:** Incident (`incident`)
  - **Type:** `onSubmit`
  - **Active:** `true`

Script Logic:function onSubmit() {

if (g_form.getValue('impact') == '1' && g_form.getValue('assigned_to') == '') {

g_form.showErrorBox('assigned_to', 'Assigned To is mandatory for High impact incidents.');

return false;

}

return true;

}

Task 5: Create `onCellEdit` Client Script (Prevent List Edit)

- **Navigation:** `System UI` → `Client Scripts` → `New`
- **Configuration:**
  - **Name:** Prevent state change via list edit
  - **Table:** Incident (`incident`)
  - **Type:** `onCellEdit`
  - **Field Name:** `State`
  - **Active:** `true`

Script Logic:function onCellEdit(sysIDs, table, oldValues, newValue, callback) {

alert('State cannot be changed from the list. Please open the incident.');

callback(false);

}

---

4. Testing & Verification

Test Case	Scenario / Action	Expected Result	Actual Result	Status
1. Mandatory Enforcement	Set `Impact` to `1 - High`, leave `Assigned To` blank, click `Submit` on `INC0010012`.	Submission blocked; error box displayed under `Assigned To`.	Script intercepted submission with message *"Assigned To is mandatory for High impact incidents."*	**Pass**

Test Case	Scenario / Action	Expected Result	Actual Result	Status
2. Successful Record Save	Set `Assigned To` to `Beth Anglin` with `Impact` as `1 - High`, click `Submit`.	Form submits successfully without client-side errors.	Record saved and redirect executed without issue.	**Pass**
3. Reverse Condition	Change `Impact` from `1 - High` to `2 - Medium`.	`Urgency` becomes editable; `Assigned To` mandatory flag clears.	Attributes reset properly, confirming `Reverse if false` behavior.	**Pass**
4. List Edit Prevention	Double-click `State` column in list view (`Incident` → `All`).	Edit cancelled; alert notification shown to user.	Alert popup rendered: *"State cannot be changed from the list..."* and cell edit rolled back.	**Pass**
5. Form-Based Update	Open `INC0008111` in Form View, change `State` to `In Progress`, click `Update`.	State updates correctly when submitted through form UI.	Form update processed successfully without script interference.	**Pass**

---

5. Conclusion

The implementation successfully demonstrates how **UI Policies** and **Client Scripts** combine to form dynamic, user-friendly, and robust client-side controls in ServiceNow. These configurations guarantee data consistency, optimize user workflows, and uphold process integrity across the entire incident lifecycle.